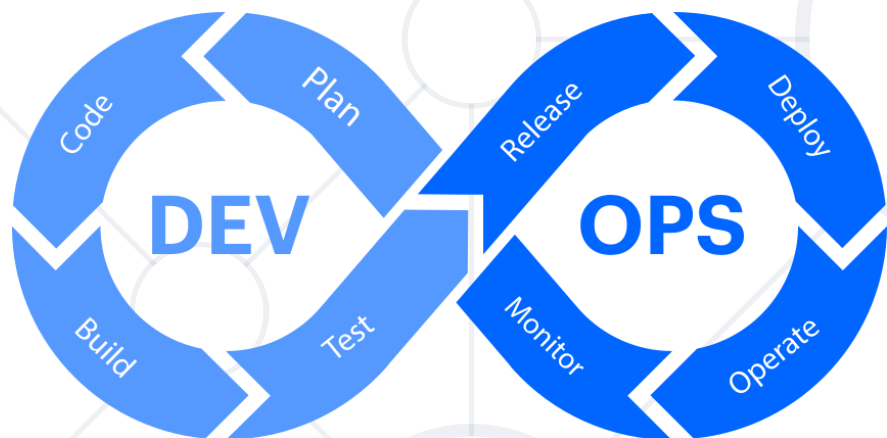


# Security Integration

From DevOps to DevSecOps



**SoftUni Team**  
**Technical Trainers**



**SoftUni**



**Software University**

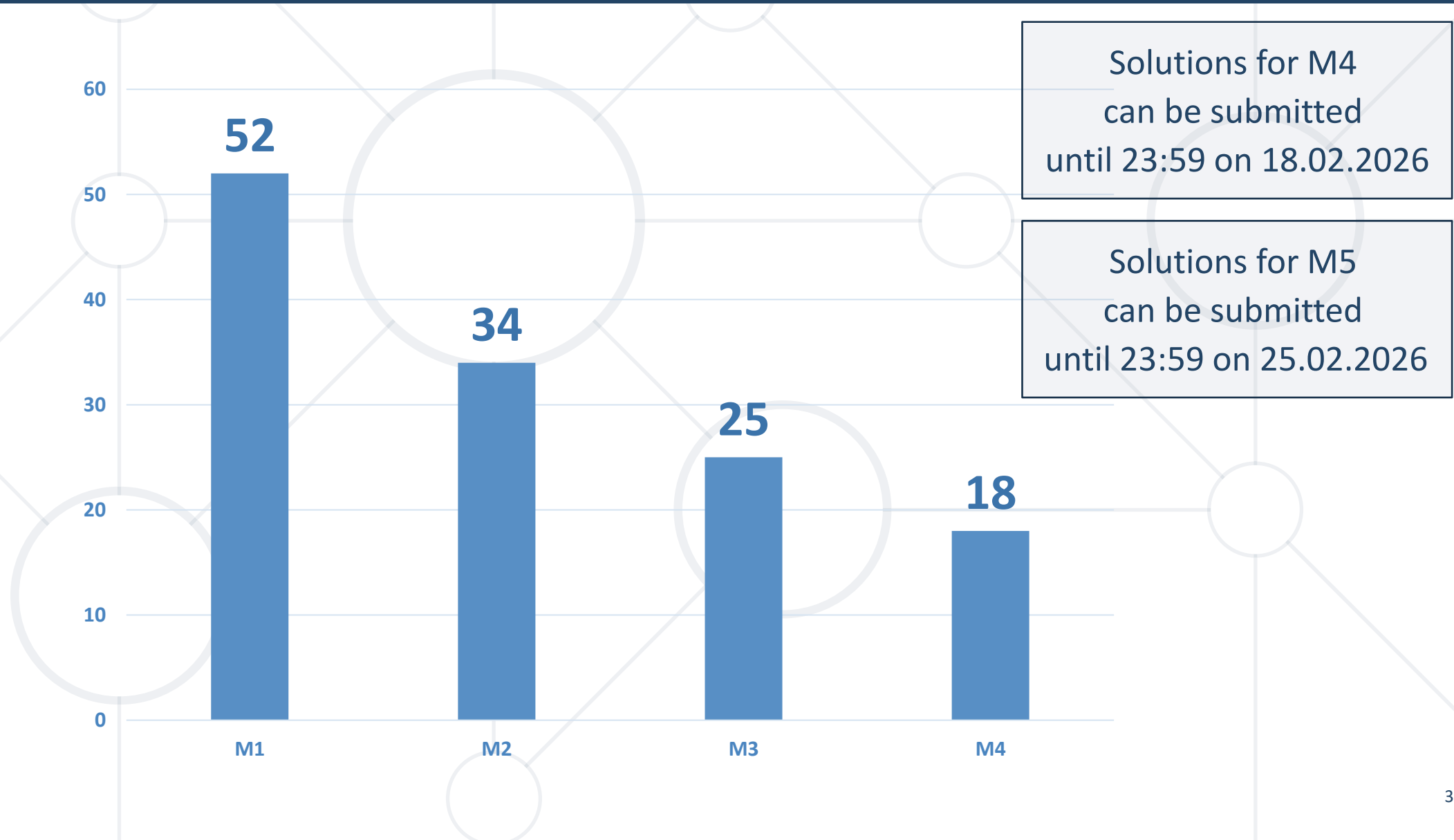
<https://softuni.bg>

[sli.do](https://sli.do)

**#DevOps-CI-CD**

[facebook.com/groups/  
cicdmonitoringjanuary2026](https://facebook.com/groups/cicdmonitoringjanuary2026)

# Homework Progress





# **Previous Module (M4)**

## **Quick Overview**

# What We Covered

- 
1. Packaging and Templating
  2. Deployment and Organizational Strategies
  3. Enable Automated Deployment

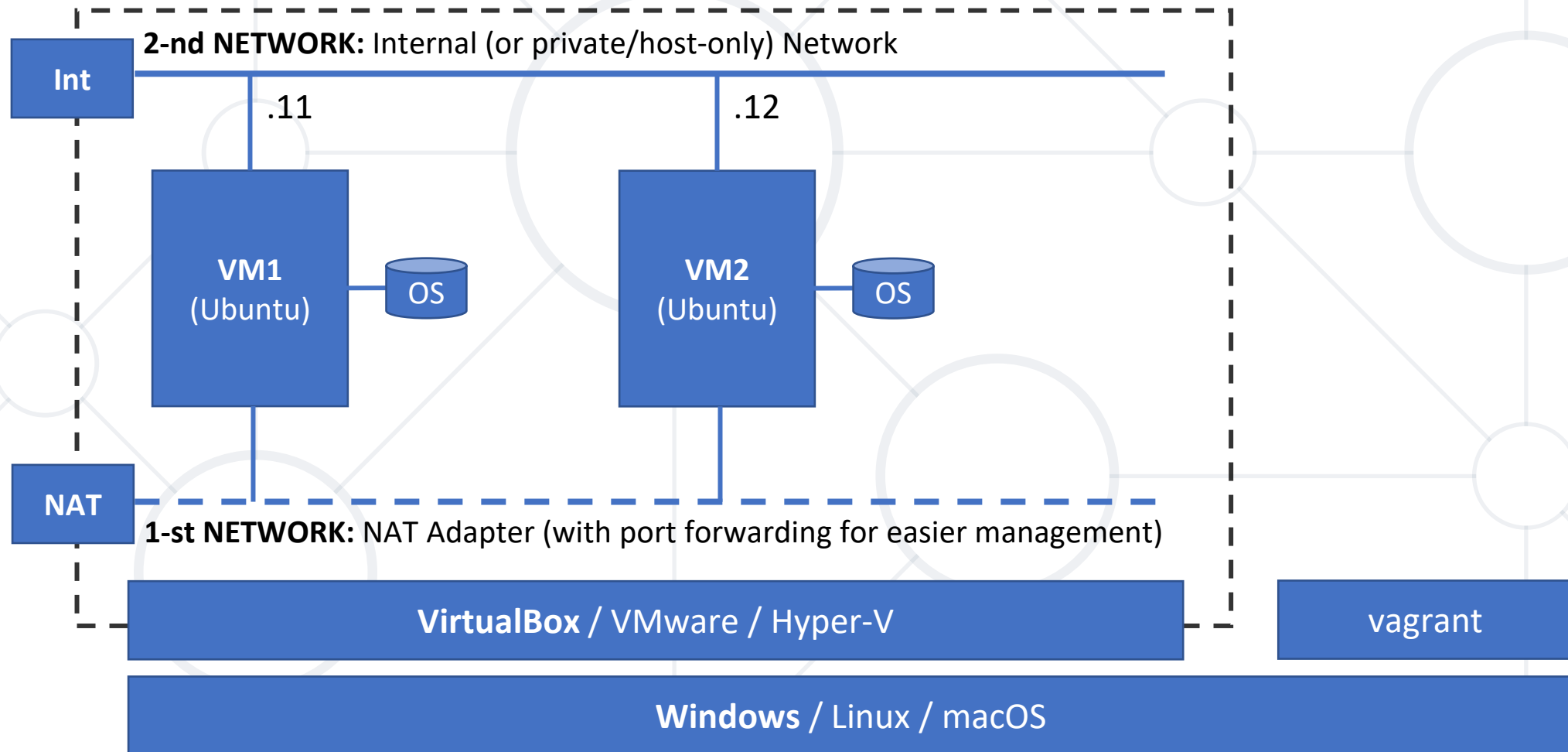


**This Module (M5)**  
**Topics and Infrastructure**

# Table of Contents

1. From DevOps to DevSecOps
2. Key Principles and Practices
3. Common Tools
4. Secure CI/CD Pipeline









**Shifting Left**  
**From DevOps to DevSecOps**

- **The Shift Left Principle**

- In traditional DevOps, security happened at the end (the Gate)
- In DevSecOps, security starts at the first line of code

- **The Cost of Delay**

- Fixing a bug in production is way more expensive than fixing it during development

- **Integration**

- Security tools must be automated, fast, and fail the build if high-risk issues are found

- **SAST (Static Application Security Testing)**
  - Scanning source code for **bad habits** (e.g., SQL injection, hardcoded creds)
  - **Linting** is a basic form of SAST
- **SCA (Software Composition Analysis)**
  - Scanning **package.json** or **requirements.txt**, or container images
  - Most modern apps are 90% third-party libraries

- Approaches
  - Decouple application components
  - Further parametrization
  - Initial secrets management
- Tools
  - Bandit
  - GoLangCI-Lint
  - HadoLint



**Practice: Change Our Approach**  
Start implementing some of the practices 😊



# **Docker Compose**

## **Linting and Secret Leaks**

- **The Pipeline Step**

- Adding a stage that fails if it finds a string that looks like an **API key** or a **Private Key**

- **Best Practice**

- Using **.gitignore** and **pre-commit hooks** to stop the leak before the git push

- Approaches
  - Reinvent Docker Compose
  - Further parametrization
  - Secrets management
- Tools
  - DCLint
  - GitLeaks





# Practice: Linting and Secret Leaks

## Let's Move Forward



# **Kubernetes & Helm**

## **Secrets Management and Hardening**

- **The Problem**

- A clean **Python app** sitting on a **dirty Alpine or Debian base image** with **50 known vulnerabilities (CVEs)**

- **The Solution**

- Scanning the image layer-by-layer after the build but ***before*** the push to the registry

- **The Tool**

- **Trivy, Gype, etc.**

- **The Concept**

- Helm charts and K8s manifests can be insecure (e.g., running as root, privileged mode, no resource limits, etc.)

- **Policy as Code**

- Using tools like **Kube-score** or **Checkov** to analyze YAML files

- **The Distinction**
  - **ConfigMaps** are for non-sensitive data
  - **Secrets** are for passwords, tokens, and certificates
- **Base64 is NOT Encryption**
  - Remember that K8s Secrets are just encoded, not encrypted by default
- **Approach**
  - Using Gitea/Jenkins **Secret Variables** to inject values into the pipeline at runtime

- **The Problem**

- We want to store everything in Git, but we can't store secrets there

- **A Solution**

- **Sealed Secrets (Bitnami)** - encrypt the secret with a public key; only the K8s cluster can decrypt it

- **A Pro Level Solution**

- **HashiCorp Vault** - A dedicated server for managing secrets with automatic rotation

- **Default Behavior**

- In K8s, every pod can talk to every other pod

- **Zero Trust**

- Implementing **NetworkPolicies** to ensure, for example, only the Frontend can talk to the API, and only the API can talk to the Database

- **The Blast Radius**

- If the Frontend is hacked, the hacker cannot move to the Database

- Approaches
  - Address changes in Kubernetes manifests and Helm charts
  - Secrets management
  - Further hardening
- Tools
  - KubeConform
  - Kube-linter
  - Trivy

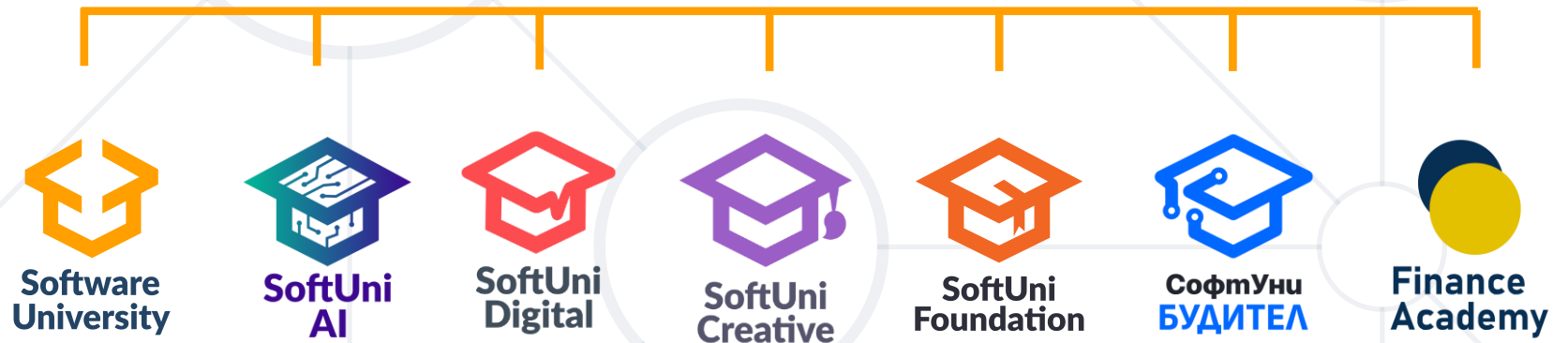




# Practice: Manifests and Charts

## Let's Move Even Further

# Questions?



- Software University – High-Quality Education, Profession and Job for Software Developers

- [softuni.bg](http://softuni.bg), [about.softuni.bg](http://about.softuni.bg)

- Software University Foundation

- [softuni.foundation](http://softuni.foundation)

- Software University @ Facebook

- [facebook.com/SoftwareUniversity](https://facebook.com/SoftwareUniversity)



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg>
- © Software University – <https://softuni.bg>

